

Умный мониторинг комфорта

Описание кейса

Создайте full-stack сервис, который превращает поток технических сообщений от датчиков в понятные выводы и помогает людям выбирать комфортное время для учёбы, встреч и отдыха.

Содержание

1	Контекст	1
2	Обязательные требования	1
2.1	Технологии	1
2.2	Импорт данных	2
2.3	Текущее состояние атриума	2
2.4	История измерений	3
2.5	Аналитика	3
2.6	Отчёты пользователей	4
2.7	REST API	4
2.8	Состояния интерфейса	5
2.9	Deployment и GitHub	5
3	Что не требуется	6
4	Бонусные возможности	6
4.1	Визуализация данных	6
4.2	Умные рекомендации	6
4.3	Работа с данными	7
4.4	Пользовательский опыт	7
5	Критерии оценивания	7
5.1	Backend, API и работа с данными — 30 баллов	7
5.2	Frontend и удобство использования — 25 баллов	8
5.3	Аналитика и решение реальной проблемы — 20 баллов	8
5.4	Креативность и дополнительные возможности — 15 баллов	8
5.5	Презентация и документация — 10 баллов	9
6	Минимальный чек-лист перед сдачей	9
7	Главный вопрос кейса	9

1 Контекст

В атриуме NU установлены датчики, которые регулярно публикуют в Telegram данные о состоянии пространства:

- температуру внутри атриума;
- температуру на улице;
- уровень освещения;
- уровень шума;
- время измерения.

Однако Telegram-канал представляет собой длинную ленту отдельных сообщений. По ней трудно быстро понять:

- комфортно ли находиться в атриуме сейчас;
- когда в атриуме прохладнее и тише;
- в какое время становится слишком жарко или шумно;
- как условия внутри отличаются от температуры на улице;
- повторяются ли проблемные периоды.

Вам предоставлен файл `result.json` с реальными сообщениями Telegram-канала за несколько дней. В нём встречаются измерения температуры внутри и снаружи, а также категории освещения и шума.

Задача команды. Создайте и разверните full-stack веб-приложение, которое превращает необработанные сообщения датчиков в понятный и полезный сервис для студентов и сотрудников университета.

Приложение должно не просто показывать исходные сообщения, а помогать пользователю понять состояние атриума и принять решение: подходит ли это место сейчас для учёбы, встречи или отдыха.

Название приложения команда выбирает самостоятельно. Название данного кейса не является обязательным названием проекта.

2 Обязательные требования

2.1 Технологии

Проект должен использовать:

- **Frontend:** React и TypeScript;
- **Backend:** Python и FastAPI;
- **База данных:** SQLite;
- **Работа с базой:** ORM;

- **Обмен данными:** REST API и JSON.

Frontend должен получать данные от backend через HTTP-запросы. Статический сайт без работающего API не считается выполненным full-stack проектом.

2.2 Импорт данных

Используйте предоставленный файл `result.json` как начальный источник данных. Данные должны быть преобразованы в структурированный вид и сохранены в базе данных.

Минимальная модель измерения должна содержать:

```
Reading
- id
- measured_at
- location
- temperature
- brightness
- noise
```

Где:

- `location` — `atrium` или `outside`;
- `brightness` может отсутствовать у измерений с улицы;
- `noise` может отсутствовать у измерений с улицы.

Импорт можно реализовать отдельным скриптом, через `seed.py` или автоматически при первом запуске приложения.

Подключение к Telegram в реальном времени **не является обязательным**.

2.3 Текущее состояние атриума

На главной странице необходимо показать последние доступные данные:

- температуру в атриуме;
- температуру на улице;
- уровень освещения;
- уровень шума;
- дату и время последнего измерения.

Также приложение должно показывать понятную текстовую оценку состояния, например:

- «Комфортно»;
- «Жарко»;
- «Очень жарко»;
- «Тихо»;

- «Шумно»;
- «Подходит для учёбы»;
- «Лучше выбрать другое место».

Эти формулировки приведены лишь как примеры: команда может создать собственную версию. Правила оценки команда определяет самостоятельно и кратко описывает в `README.md`.

Оценки являются частью логики приложения и не должны представляться как официальные медицинские или санитарные рекомендации.

2.4 История измерений

Пользователь должен иметь возможность просматривать историю данных.

Обязательно реализуйте:

- список или таблицу измерений;
- фильтр по дате;
- фильтр по месту: атриум или улица;
- минимум один дополнительный фильтр:
 - по уровню шума;
 - по освещению;
 - по температурному диапазону;
- сортировку по времени;
- сортировку по температуре.

Каждая запись должна содержать время, место, температуру и доступные характеристики среды.

2.5 Аналитика

Приложение должно вычислять показатели, которых нет в готовом виде в исходном JSON. Обязательно покажите:

1. минимальную, максимальную и среднюю температуру за выбранный день;
2. минимум **один собственный аналитический вывод**.

Примеры собственного вывода:

- самое прохладное время;
- самое тихое время;
- лучший период для учёбы;
- самый жаркий период дня;
- количество некомфортных измерений;

- разница между температурой внутри и снаружи;
- «оценка комфорта» по собственной формуле.

Список приведён лишь как ориентир: команда свободна предложить собственный показатель и должна уметь объяснить, как он рассчитывается.

2.6 Отчёты пользователей

Добавьте возможность оставить отзыв о текущих условиях в атриуме. Пользователь должен иметь возможность выбрать категорию:

- слишком жарко;
- слишком шумно;
- слишком ярко;
- слишком темно;
- комфортно;
- другое.

Отчёт должен сохраняться в базе данных. Минимальная модель:

```
Report
- id
- created_at
- category
- comment
- status
```

Для отчётов необходимо реализовать полный CRUD:

- создание;
- просмотр;
- редактирование;
- удаление.

Поле **status** может содержать, например, **open** или **resolved**. Авторизация пользователей для этого кейса не требуется.

2.7 REST API

Backend должен содержать как минимум следующие возможности:

Метод	Маршрут
GET	/api/readings
GET	/api/readings/{id}
GET	/api/summary
GET	/api/reports
POST	/api/reports
PUT или PATCH	/api/reports/{id}
DELETE	/api/reports/{id}

Названия маршрутов могут отличаться, если структура API остаётся понятной и логичной.

API должно:

- возвращать корректные HTTP-коды;
- обрабатывать ситуацию, когда запись не найдена;
- проверять входящие данные;
- возвращать JSON.

2.8 Состояния интерфейса

Frontend должен корректно показывать:

- загрузку данных;
- состояние ошибки (error state);
- состояние отсутствия данных (empty state);
- успешное создание или изменение отчёта.

Интерфейс должен оставаться понятным на экране ноутбука и телефона.

2.9 Deployment и GitHub

Проект необходимо разместить в открытом GitHub-репозитории. Также должны быть доступны:

- публичная ссылка на frontend;
- публичная ссылка на backend.

Рекомендуемые сервисы для деплоя:

- frontend — Vercel;
- backend — Railway.

В README.md укажите:

- описание проекта;
- ссылки на развёрнутое приложение;

- инструкции для локального запуска;
- структуру проекта;
- правила расчёта статуса или comfort score;
- реализованные обязательные и бонусные функции.

3 Что не требуется

Для выполнения основного задания не нужны:

- регистрация и авторизация;
- административная панель;
- получение данных в реальном времени;
- машинное обучение.

Сначала завершите работающий MVP. Дополнительные функции оцениваются только после выполнения обязательных требований.

4 Бонусные возможности

Команда может выбрать любые дополнительные функции. Количество не ограничено, но обязательные требования остаются приоритетом.

4.1 Визуализация данных

- график изменения температуры;
- сравнение температуры внутри и снаружи;
- график шума и освещения;
- тепловая карта по часам;
- переключение между днями.

4.2 Умные рекомендации

- «Лучшее время для учёбы»;
- «Лучшее время для встречи»;
- прогноз состояния на основе предыдущих часов;
- автоматическое обнаружение необычных измерений;
- предупреждение о резком изменении температуры;
- собственный comfort score.

4.3 Работа с данными

- автоматический парсер Telegram-экспорта;
- загрузка нового JSON-файла через интерфейс;
- экспорт измерений в CSV;
- дополнительные API-фильтры;
- пагинация;
- объединение близких по времени измерений внутри и снаружи.

4.4 Пользовательский опыт

- светлая и тёмная темы;
- доступный интерфейс и управление с клавиатуры;
- разные цветовые состояния для комфортных и проблемных условий;
- dashboard для обработки пользовательских отчётов;
- отметка отчёта как решённого;
- подтверждение перед удалением;
- понятные уведомления об успешных действиях.

5 Критерии оценивания

Максимальная оценка — 100 баллов

5.1 Backend, API и работа с данными **30 баллов**

Оценивается:

- корректный импорт данных;
- понятная структура базы;
- использование ORM;
- работающие REST endpoint'ы;
- полный CRUD отчётов;
- валидация данных;
- обработка ошибок;
- качество и структура backend-кода.

Высокую оценку получает проект, в котором данные действительно обрабатываются на сервере, а не хранятся только во frontend.

5.2 Frontend и удобство использования **25 баллов**

Оценивается:

- понятность главной страницы;
- отображение текущего состояния;
- качество истории измерений;
- фильтры и сортировка;
- форма пользовательского отчёта;
- состояния загрузки, ошибки и пустого результата;
- адаптивность;
- общее удобство интерфейса.

Дизайн не обязан быть сложным, но должен помогать пользователю быстро понять данные.

5.3 Аналитика и решение реальной проблемы **20 баллов**

Оценивается:

- правильность вычислений;
- полезность аналитических показателей;
- логичность правил определения комфорта;
- способность приложения отвечать на реальные вопросы пользователей;
- объяснимость выбранных алгоритмов.

Простой красивый dashboard без полезной интерпретации данных получит меньше баллов, чем приложение, которое действительно помогает принять решение.

5.4 Креативность и дополнительные возможности **15 баллов**

Оценивается:

- оригинальная идея;
- собственная «изюминка»;
- качественно реализованные бонусные функции;
- интересная визуализация;
- нестандартное, но понятное использование данных.

Количество бонусных функций менее важно, чем качество их реализации.

5.5 Презентация и документация 10 баллов

Оценивается:

- понятное объяснение проблемы и решения;
- живое демо;
- способность объяснить архитектуру и код;
- качество README.md;
- готовность ответить на вопросы жюри.

6 Минимальный чек-лист перед сдачей

Проект готов к сдаче, если:

- ☐ данные из `result.json` отображаются через backend API;
- ☐ измерения сохранены в SQLite;
- ☐ показано текущее состояние атриума;
- ☐ работает история с фильтрацией и сортировкой;
- ☐ рассчитаны обязательные статистические показатели;
- ☐ реализован минимум один собственный аналитический вывод;
- ☐ отчёты пользователей можно создать, посмотреть, изменить и удалить;
- ☐ frontend и backend развёрнуты;
- ☐ GitHub-репозиторий открыт;
- ☐ заполнен README.md;
- ☐ команда готова показать живое демо.

7 Главный вопрос кейса

Как превратить поток технических сообщений от датчиков в понятный сервис, который помогает людям выбирать комфортное время и место для учёбы, встреч и отдыха?

Необязательно реализовать всё возможное. Сначала создайте надёжный, понятный и работающий продукт, а затем добавляйте собственные идеи.

Удачи на хакатоне!